



Université Montpellier Paul-Valéry

Master MIASHS

Mémoire de stage

---

# **Polypbase**

## **Conception d'une webapp de suivi des polypes de méduses**

Aquarium de Paris

Suivi de laboratoire, données historiques et interface tablette

---

Réalisé par

Ayoub AKKOUH,  
Anthony COMBES-AGUÉRA,  
et Clément ROMANET

---

Tuteur universitaire

Sophie LEBRE

Tuteurs de stage

Étienne BOURGOUIN

Anaïs COURTET

---

Dépôt GitHub

Anthony2210/POLYPBASE

Année universitaire 2025-2026

## Remerciements

Nous tenons à remercier Anaïs Courtet et Étienne Bourgoïn pour leur accueil au sein de l'Aquarium de Paris et pour le temps qu'ils nous ont accordé depuis le début du stage. Leur disponibilité, leurs explications et leurs retours nous ont permis de mieux comprendre le fonctionnement du service des méduses et les besoins réels derrière Polybase. Nous les remercions aussi pour leur sympathie et pour la visite guidée de l'Aquarium. Cette découverte du lieu, des espaces ouverts au public et des coulisses nous a aidés à mieux situer notre travail. Polybase accompagne un travail quotidien en laboratoire, dans un environnement vivant, précis et déjà très organisé. Nous remercions Madame Sandra Bringay pour son aide sur la reprise du modèle conceptuel de données et pour les nombreux renseignements qu'elle nous a apportés sur le projet. Ses retours nous ont aidés à mieux structurer les entités importantes et à prendre plus de recul sur la manière de modéliser les données. Nous remercions également Monsieur Servajean pour ses conseils sur l'usage de Git, des branches et de l'organisation du dépôt. Ces remarques nous ont permis de corriger notre manière de travailler et de préparer une base plus propre pour la suite du stage. Enfin, nous remercions Madame Sophie Lebre, qui nous a transmis cette offre de stage. Son partage nous a permis de rejoindre un projet concret, utile et directement lié à des problématiques de données, de suivi scientifique et de développement web.

# Table des matières

|       |                                                                 |    |
|-------|-----------------------------------------------------------------|----|
| 1.    | Introduction . . . . .                                          | 2  |
| 2.    | Contexte de l’Aquarium de Paris et du service méduses . . . . . | 3  |
| 3.    | État de l’art . . . . .                                         | 4  |
| 4.    | Problématique et objectifs du stage . . . . .                   | 5  |
| 5.    | Organisation du projet et méthode de travail . . . . .          | 6  |
| 6.    | Analyse des données historiques . . . . .                       | 7  |
| 6.1.  | Structure des fichiers source . . . . .                         | 7  |
| 6.2.  | Création du CSV complet . . . . .                               | 7  |
| 6.3.  | Contrôles de qualité des données . . . . .                      | 8  |
| 6.4.  | Périmètre du suivi . . . . .                                    | 9  |
| 6.5.  | Premiers constats exploitables . . . . .                        | 10 |
| 7.    | Conception de Polybase . . . . .                                | 13 |
| 7.1.  | Choix d’architecture . . . . .                                  | 13 |
| 7.2.  | Modèle de données . . . . .                                     | 13 |
| 7.3.  | Interface et maquette . . . . .                                 | 14 |
| 8.    | Réalisations techniques . . . . .                               | 15 |
| 8.1.  | Backend Django et API . . . . .                                 | 15 |
| 8.2.  | Interface React . . . . .                                       | 15 |
| 8.3.  | Traduction, données de test et outillage . . . . .              | 16 |
| 9.    | Confidentialité, durabilité et impact de l’outil . . . . .      | 17 |
| 10.   | Bilan critique et perspectives . . . . .                        | 18 |
| 10.1. | Limites actuelles . . . . .                                     | 18 |
| 10.2. | Recul sur l’interface et les capteurs . . . . .                 | 18 |
| 10.3. | Perspectives . . . . .                                          | 18 |
| 11.   | Conclusion . . . . .                                            | 19 |
| A     | Maquette papier . . . . .                                       | 20 |

# 1. Introduction

Notre travail part d'un besoin très concret de l'Aquarium de Paris. Le service des méduses suit des cultures de polypes dans le temps, avec des espèces, des souches, des boîtes, des températures et des observations régulières. Ce suivi existe déjà sous forme de fichiers Excel. Ces fichiers ont une valeur importante, car ils conservent plusieurs années d'observations. Leur forme rend cependant la recherche, l'historisation, le partage et la reprise difficiles dès que le nombre de boîtes augmente ou que plusieurs personnes interviennent. La première ambition de Polypbase est de créer un outil commun entre plusieurs aquariums. Chaque structure doit pouvoir conserver ses propres données, tout en utilisant une manière partagée de nommer les espèces, de noter les boîtes, de saisir les relevés et d'échanger certaines informations. Cette cohérence est essentielle pour que les données puissent être comprises ailleurs qu'au laboratoire qui les a produites. Elle demande une gestion des organisations, des droits, des espèces, des souches, des transferts et des règles de partage. Le stage consiste à transformer ce suivi dispersé en une application web utilisable au laboratoire et sur ordinateur. L'outil doit permettre de retrouver rapidement une boîte, de saisir un relevé, de consulter l'historique, de suivre les zones thermiques, de préparer des exports et de garder une trace des actions. La boîte de culture reste le point de repère principal. Elle relie l'espèce, la souche, la localisation, la température, les relevés, les observations, les repiquages et les échanges éventuels entre structures. Ce mémoire présente le contexte scientifique et professionnel du stage, puis l'état de l'art utilisé pour cadrer le projet. Il expose ensuite la problématique, l'organisation du travail, l'analyse des fichiers historiques, la conception de Polypbase, les réalisations techniques, les questions de confidentialité et de durabilité, puis les limites et perspectives du projet.

## 2. Contexte de l’Aquarium de Paris et du service méduses

L’Aquarium de Paris est un lieu de présentation, de médiation et de travail scientifique autour du monde aquatique. Situé dans les Jardins du Trocadéro, il accueille le public tout en maintenant en coulisses des espaces techniques et des élevages spécialisés. Son Médusarium occupe une place importante dans cette identité. L’établissement présente une grande collection de méduses en Europe, avec des espèces montrées par roulement, et ses biologistes travaillent sur l’entretien, le maintien et la reproduction de ces animaux [1]. Le stage se déroule au contact du service des méduses. Ce service suit des cultures de polypes dans le temps. Pour comprendre l’intérêt de Polypbase, il faut rappeler le cycle que l’application accompagne. La méduse correspond au stade libre et visible de l’animal. Le polype est un stade fixé, plus discret, qui peut rester dans une boîte de culture et produire de nouveaux individus. L’éphyrule est le jeune stade libéré lors du passage du polype vers la forme méduse. Dans la littérature scientifique, on parle souvent d’éphyra. Dans le laboratoire et dans les fichiers de suivi, le terme éphyrule est utilisé pour le comptage quotidien. Le passage du polype vers l’éphyrule est appelé strobilation. Helm décrit ce processus comme la métamorphose d’un polype en jeune méduse, avec une production possible d’une ou plusieurs éphyras selon les espèces [5]. Cette étape explique pourquoi le suivi doit dépasser le simple nombre de boîtes. Il faut savoir quelle souche est observée, dans quelle zone thermique elle se trouve, combien de polypes sont présents, combien d’éphyrules sont produites et quels événements ont eu lieu autour de la boîte. Les cultures suivies correspondent à des souches et à des espèces différentes. Elles sont placées dans des boîtes, puis maintenues dans des conditions contrôlées, notamment avec des températures différentes selon les besoins du suivi. Le rôle de l’équipe est d’observer régulièrement ces boîtes, de compter les polypes, de relever les éphyrules et de noter les événements importants. Ces informations servent à comprendre l’évolution des cultures, à garder un historique fiable et à préparer les décisions de laboratoire.

### 3. État de l’art

Le projet s’inscrit dans plusieurs familles de références. La première est celle de la numérisation d’un suivi de laboratoire. L’objectif est de rendre les données plus traçables, de conserver leur historique, de retrouver rapidement une boîte et de préparer une réutilisation scientifique ou réglementaire des relevés. La deuxième référence est celle des plateformes existantes pour aquariums. Fauniabase, développé par Luxaqua, est présenté comme un logiciel de gestion et de suivi des animaux pour les aquariums publics. Il met en avant la centralisation des données, le suivi des animaux et des bacs, la traçabilité, la planification de tâches et l’usage de QR codes [3]. Cette référence montre l’existence d’outils professionnels autour de la gestion d’inventaires vivants en aquarium. Fauniabase apporte une référence utile pour Polybase. Polybase cible toutefois un besoin plus spécialisé : les cultures de polypes de méduses, leurs relevés réguliers, leur historique, les repiquages, les conditions de température et l’usage en laboratoire sur tablette. Le travail retient surtout certaines idées utiles, comme la centralisation des données, la traçabilité, la recherche rapide et l’identification par QR code. Une autre plateforme présentée pendant le cadrage a surtout rappelé les difficultés à éviter. Elle rassemblait beaucoup d’informations avec un usage lourd, une navigation complexe, de nombreuses références et plusieurs termes pour désigner des objets proches. Ce type d’outil montre le risque principal pour Polybase. Dans un laboratoire, l’utilisateur doit pouvoir saisir un relevé sans devoir comprendre une architecture compliquée. L’application doit utiliser des codes clairs, des écrans courts et des actions évidentes. La troisième référence concerne la taxonomie. Une application utilisée par plusieurs structures doit éviter les ambiguïtés sur les noms d’espèces. Le World Register of Marine Species, ou WoRMS, donne accès à des noms valides, des synonymes, des noms vernaculaires et des identifiants stables pour les organismes marins [4]. Polybase peut ainsi relier ses espèces à des références reconnues sans porter toute la complexité taxonomique. C’est pour cette raison que le modèle **Species** prévoit un identifiant WoRMS éventuel. La quatrième référence vient de la biologie des méduses. Les articles sur les polypes et les éphyrales aident à justifier les données suivies. Helm rappelle que beaucoup de scyphozoaires ont un cycle comprenant une larve planula, un polype benthique et une méduse, et que la transformation du polype en éphyra s’appelle la strobilation [5]. Fuchs et ses coauteurs montrent aussi que, chez *Aurelia aurita*, la transition du polype vers la méduse est déclenchée par des signaux environnementaux et que des protéines sont fortement activées avant la métamorphose en réponse aux changements saisonniers de température [6]. La température apparaît donc comme une donnée importante à conserver. Une étude expérimentale sur trois espèces de scyphozoaires indique que la température peut jouer un rôle dans la reproduction asexuée des polypes, le moment de la strobilation et la production d’éphyras [7]. Ces résultats expliquent l’intérêt de relier les comptages biologiques aux zones thermiques, aux sondes, aux changements de température et aux historiques de boîtes. Le contexte scientifique de l’Aquarium donne aussi du sens au projet. Le LabCom Jellyfish and Polyps Academy explique que l’Aquarium de Paris maintient une grande collection de polypes de méduses et travaille avec MARBEC sur la culture des méduses, la création d’une banque de référence de polypes et la compréhension des paramètres qui influencent leurs proliférations [2]. Polybase s’inscrit dans cette logique. L’application doit aider à conserver des données plus propres, plus faciles à retrouver et plus adaptées à une analyse future.

## 4. Problématique et objectifs du stage

La problématique du stage peut se formuler ainsi : comment concevoir une application web maintenable capable de remplacer un suivi Excel historique, avec une saisie simple en laboratoire, une consultation claire sur ordinateur et une base commune utilisable par plusieurs aquariums. Cette problématique contient plusieurs contraintes. La première est métier. L'application doit respecter la façon dont le service des méduses suit ses cultures. Elle doit donc intégrer les boîtes, les souches, les espèces, les températures, les observations, les repiquages, les éphyrules et l'historique. La deuxième contrainte est ergonomique. Une tablette utilisée en laboratoire doit afficher peu d'informations et permettre une saisie rapide. La troisième contrainte est technique. Le projet doit rester compréhensible par des développeurs qui reprendront le code après le stage. La quatrième contrainte concerne la donnée. Les fichiers Excel anciens doivent être compris avant toute tentative d'import. Nous avons avancé par étapes. Il fallait d'abord cadrer le besoin et comprendre les données existantes. Il fallait ensuite structurer le projet avec une architecture claire, créer les premiers modèles Django, stabiliser une API propre et produire une première interface React orientée tablette et ordinateur. La suite du travail prépare aussi les données de démonstration, la documentation, le notebook d'analyse exploratoire et les scripts d'import ou de QR codes à compléter. Le projet reste en construction. Le travail réalisé pose une base technique et méthodologique. Les imports réels, les QR codes, les exports, l'administration métier complète et l'intégration des sondes doivent encore être finalisés.

## 5. Organisation du projet et méthode de travail

Nous avons organisé le travail autour d'échanges réguliers avec l'équipe de l'Aquarium de Paris. Anaïs Courtet et Étienne Bourgouin nous ont accompagnés dans la compréhension du besoin métier. Ces échanges ont permis de clarifier les contraintes du laboratoire, de corriger certains choix de conception et de vérifier que les premiers écrans restaient cohérents avec l'usage attendu. Le projet a été organisé autour d'un dépôt GitHub public. Le code est séparé en dossiers fonctionnels afin de faciliter la lecture et la reprise. Le dossier **backend** contient Django, les modèles, les API et les tests. Le dossier **frontend** contient l'application React. Le dossier **docs** contient les notes, les maquettes et le MCD. Le dossier **notebooks** contient l'analyse exploratoire des fichiers Excel. Le dossier **scripts** contient les points d'entrée prévus pour l'import Excel et la génération de QR codes. Un espace Notion a aussi été utilisé pour garder les décisions, les commandes utiles et les notes de travail. Ce support facilite la reprise par les membres du groupe et par les personnes qui interviendront après le stage. Le README complète cette organisation en expliquant les commandes, la structure du projet, les endpoints API, les comptes de démonstration et la gestion des dépendances avec uv. La méthode suivie a consisté à construire d'abord la base du projet avant de pousser l'interface trop loin. Le modèle de données et l'API ont été priorisés, car le frontend dépend directement de la structure des données. L'interface React a ensuite servi à tester les usages principaux : recherche d'une boîte, consultation d'une fiche, saisie d'un relevé, affichage des zones thermiques et modification de la langue du profil. Cette organisation répond aussi à une contrainte de maintenance. Le tuteur de stage a indiqué que l'Aquarium pourra faire appel à des informaticiens pour la reprise. Le code doit donc rester lisible, commenté avec des phrases simples en anglais, et documenté suffisamment pour qu'un développeur extérieur comprenne rapidement le rôle de chaque partie.

## 6. Analyse des données historiques

### 6.1. Structure des fichiers source

Les fichiers de suivi historiques montrent comment les données sont réellement notées par l'équipe. Les fichiers `Suivi_2019.xlsx` à `Suivi_2026.xlsx` sont organisés par années. Dans ces fichiers, les feuilles correspondent à de grands groupes comme Aurelia, Hydrozoa, Cubozoa, Rhizostomae, Semaestomeae ou Coronatae. Certaines années ajoutent aussi d'autres feuilles, comme UMR\_MARBEC ou Staurozoa. Dans les feuilles annuelles, les premières colonnes décrivent l'espèce, le numéro de boîte et la température. Les colonnes suivantes correspondent aux semaines ou aux mois de suivi. Pour une même boîte, les données sont notées sur plusieurs lignes, avec une ligne pour le nombre de polypes et une autre pour le nombre d'éphyrules. La collecte suit donc une logique de tableau de laboratoire. L'équipe observe les boîtes, note les comptages dans la cellule correspondant à la période de suivi, puis conserve l'historique dans le même fichier annuel. Cette forme facilite la saisie dans Excel semaine après semaine. Elle devient cependant plus difficile à exploiter quand il faut retrouver une boîte précise, comparer plusieurs années ou préparer un import dans une base de données. Le dossier de données contient aussi des fichiers complémentaires. Un fichier est consacré aux repiquages et aux changements de température. Le fichier `SuiviStrobilation.xlsx` contient des informations liées à la strobilation, avec des colonnes comme l'espèce, la température, la salinité et des conditions expérimentales. Le fichier `A.limbata.xlsx` reprend une structure plus déjà normalisée, avec une colonne de date et des groupes de colonnes associant une boîte, les polypes, les éphyrules et la température. Ces différences demandent un import plus fin qu'un simple copier coller. Il faudra reconnaître plusieurs formes de fichiers.

### 6.2. Création du CSV complet

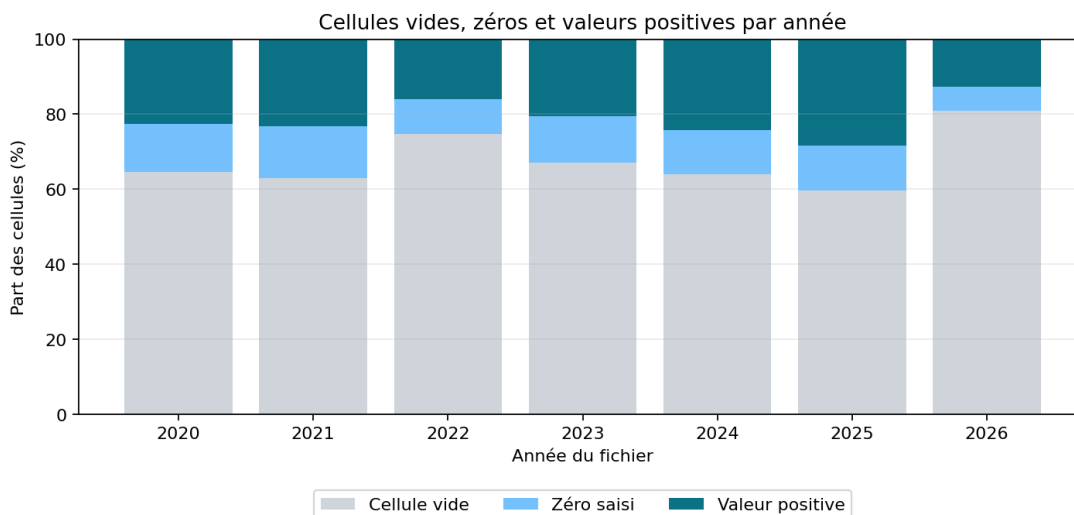
Nous avons créé le fichier `polyp_tracking_long_format.csv` avec le notebook afin de rassembler toutes les mesures dans un seul tableau. Dans ce format long, chaque ligne correspond à une mesure possible pour une boîte, une année, une semaine et un type de mesure. La colonne `file_year` vient du nom du fichier annuel. La colonne `group` vient de la feuille Excel. Les colonnes `species`, `box` et `temperature` reprennent les informations associées à la boîte. La colonne `measurement_type` conserve le libellé présent dans Excel, tandis que `normalized_measurement_type` simplifie ce libellé en deux familles principales, `polyps` et `ephyrae`. La colonne `week_index` indique la semaine reconstruite, `excel_column` garde la position de la colonne source et `value` contient la valeur numérique quand elle existe. Le CSV garde les cellules vides comme des absences de valeur. Cette décision est importante pour ne pas confondre une case vide avec un zéro saisi. Le zéro indique qu'un relevé a été fait avec une valeur nulle. Une cellule vide peut venir d'une absence de suivi, d'une boîte non active, d'un changement de température ou d'un repiquage. Ce fichier sert de support de travail pour comprendre l'import futur. Il ne remplace pas encore les modèles Django, mais il rapproche les données historiques d'une structure exploitable par l'application.

| Variable                                 | Description                                                                                 |
|------------------------------------------|---------------------------------------------------------------------------------------------|
| <code>file_year</code>                   | Année du fichier Excel source, extraite du nom du fichier.                                  |
| <code>group</code>                       | Nom de la feuille Excel, souvent lié à un groupe de méduses.                                |
| <code>species</code>                     | Nom de l'espèce écrit dans la feuille de suivi.                                             |
| <code>box</code>                         | Identifiant de la boîte dans le fichier Excel.                                              |
| <code>temperature</code>                 | Température indiquée dans la feuille Excel pour la boîte ou la ligne suivie.                |
| <code>measurement_type</code>            | Libellé original de la mesure, par exemple nombre de polypes ou nombre d'éphyrales.         |
| <code>normalized_measurement_type</code> | Type de mesure simplifié pour l'analyse, avec <code>polyps</code> ou <code>ephyrae</code> . |
| <code>week_index</code>                  | Numéro de semaine reconstruit à partir des colonnes du tableau Excel.                       |
| <code>excel_column</code>                | Position de la colonne source dans le fichier Excel.                                        |
| <code>value</code>                       | Valeur numérique saisie, laissée vide quand aucune valeur n'est présente dans Excel.        |

**Table 1** – Variables du fichier `polyp_tracking_long_format.csv`.  
Source : table construite à partir de `notebooks/excel_eda.ipynb`.

### 6.3. Contrôles de qualité des données

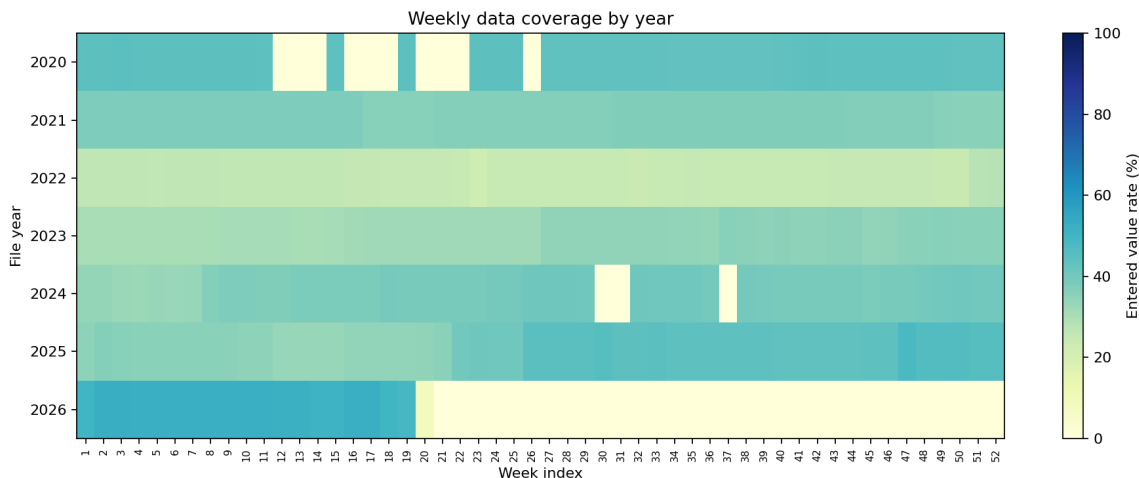
Le notebook `notebooks/excel_eda.ipynb` documente cette première analyse exploratoire. Son rôle est de comprendre la structure des fichiers, d'identifier les colonnes, de repérer les valeurs réellement saisies et de préparer une transformation utilisable par l'application. Il insiste aussi sur un point important. Une cellule vide peut avoir un sens différent de zéro. Elle peut correspondre à une absence de suivi, à une boîte inactive, à une période avant ou après un changement de température, ou à un repiquage qui a modifié le suivi. Un zéro saisi et une absence de valeur portent donc deux informations différentes.



**Figure 1** – Répartition des cellules vides, des zéros saisis et des valeurs positives par année.  
Source : figure générée depuis `notebooks/excel_eda.ipynb`.

La figure montre que les cellules vides occupent une part importante des tableaux. Ce constat oriente directement la conception de l'import. Il ne faudra pas transformer automatiquement ces vides en zéros. Le notebook produit aussi une figure de couverture hebdomadaire. Elle permet de repérer les périodes où les

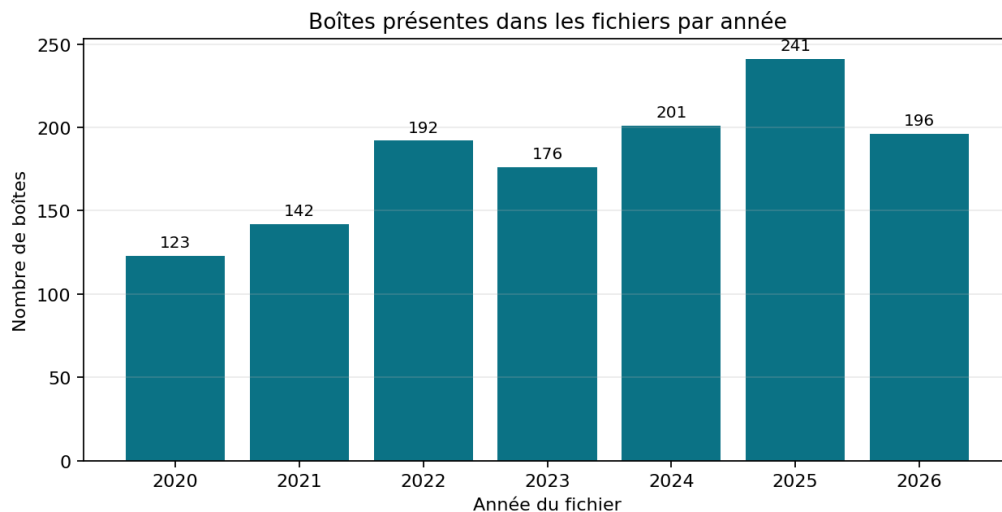
fichiers sont moins remplis et les années qui ne couvrent pas toutes les semaines de la même façon.



**Figure 2** – Taux de valeurs saisies par semaine et par année.  
Source : figure générée depuis `notebooks/excel_eda.ipynb`.

#### 6.4. Périmètre du suivi

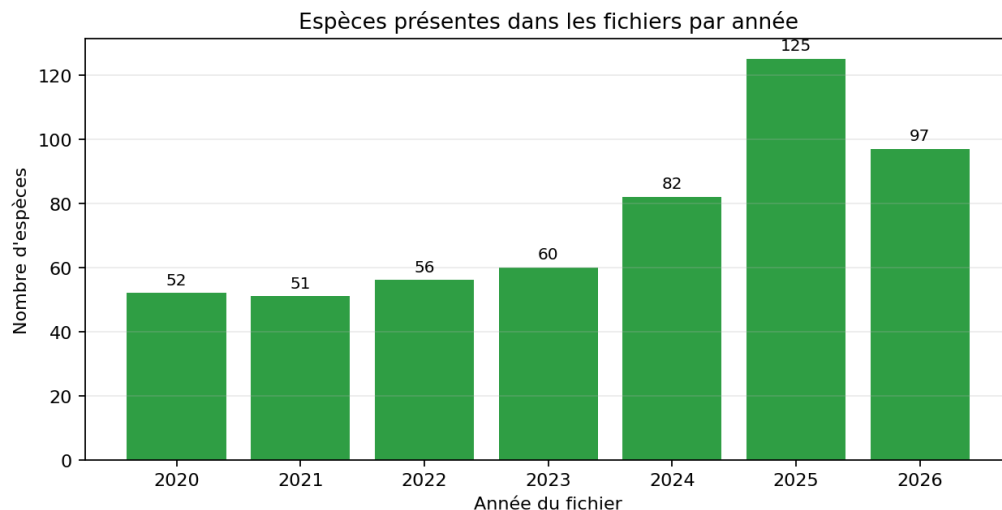
Les figures issues du notebook montrent aussi que le périmètre de suivi varie selon les années. Le nombre de boîtes présentes dans les fichiers augmente jusqu'en 2025, puis diminue dans le fichier 2026 disponible au moment de l'analyse. Cette observation aide à rester prudent dans les comparaisons entre années. Une variation peut venir d'un changement biologique, mais aussi d'une différence de périmètre, de durée de suivi ou de saisie.



**Figure 3** – Nombre de boîtes présentes dans les fichiers de suivi par année.  
Source : figure générée depuis `notebooks/excel_eda.ipynb`.

Le nombre d'espèces présentes varie aussi selon les fichiers. Ce point justifie l'importance d'une gestion claire des espèces et des souches dans Polybase. Une application multi-structures doit éviter de mélanger une

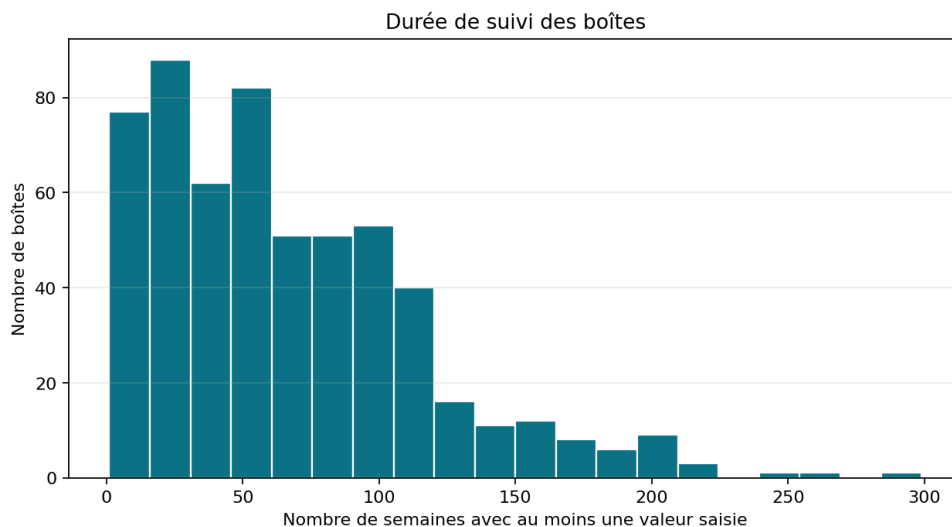
variation réelle du suivi avec une variation liée à la manière dont les noms sont écrits ou organisés dans les fichiers.



**Figure 4** – Nombre d'espèces présentes dans les fichiers de suivi par année.  
Source : figure générée depuis `notebooks/excel_eda.ipynb`.

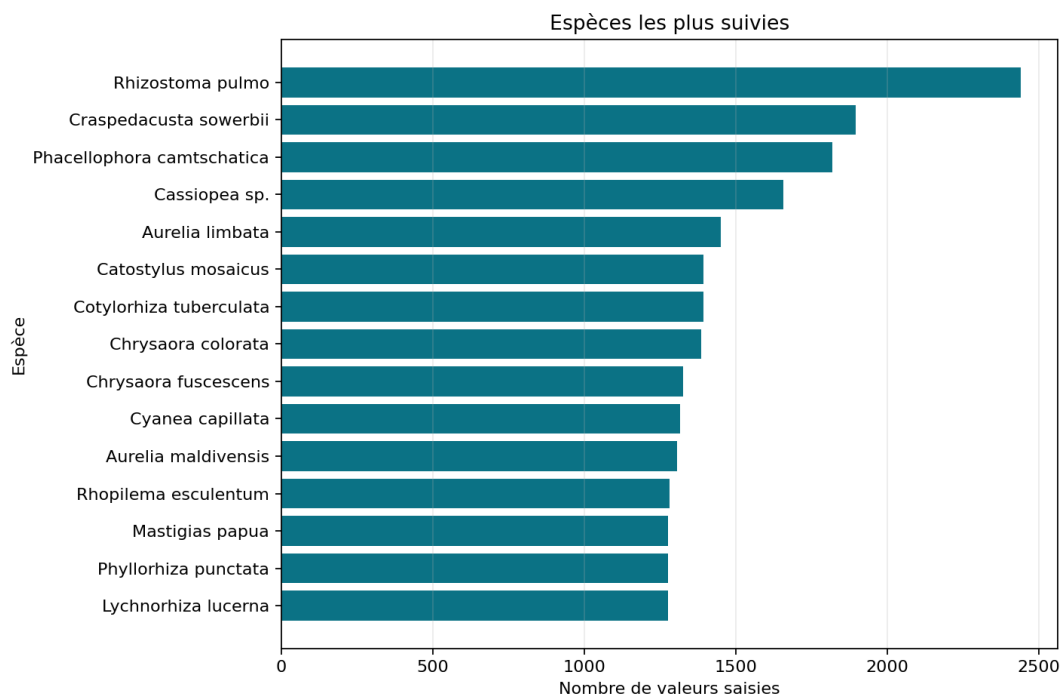
## 6.5. Premiers constats exploitables

L'analyse ne sert pas seulement à préparer un import technique. Elle aide aussi à comprendre quels écrans seront utiles dans l'application. La durée de suivi des boîtes montre que certaines cultures apparaissent sur de nombreuses semaines. Cela confirme l'intérêt d'une fiche boîte avec un historique accessible rapidement.



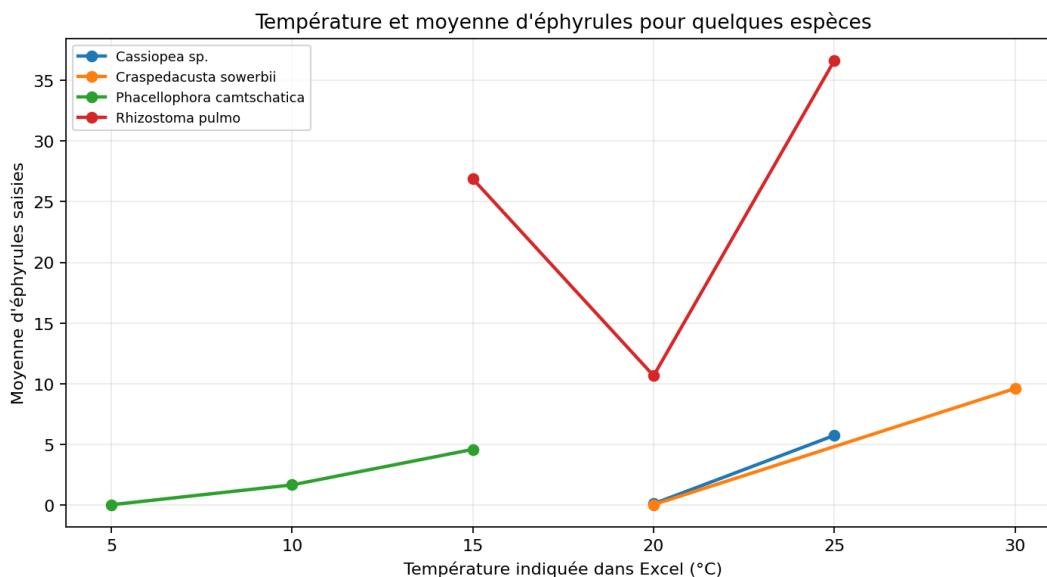
**Figure 5** – Durée de suivi des boîtes, estimée à partir des semaines avec une valeur saisie.  
Source : figure générée depuis `notebooks/excel_eda.ipynb`.

Le notebook repère aussi les espèces les plus suivies. Cette figure ne dit pas quelles espèces sont les plus importantes biologiquement. Elle montre plutôt quelles espèces pèsent le plus dans les fichiers disponibles. Cette information est utile pour choisir les premiers cas de test de l'import et de l'interface.



**Figure 6** – Espèces les plus représentées dans les valeurs saisies.  
Source : figure générée depuis `notebooks/excel_eda.ipynb`.

Une dernière lecture exploratoire compare la température indiquée dans Excel et la moyenne d'éphyrules pour quelques espèces bien représentées. Cette figure doit être interprétée avec prudence. Les données ne suffisent pas à isoler un effet biologique de la température, car les espèces, les boîtes, les périodes et les conditions de suivi changent aussi. Elle montre cependant pourquoi Polybase doit relier les relevés à une zone thermique, à des changements de température et à un historique de boîte.



**Figure 7** – Lien exploratoire entre température indiquée et moyenne d'éphyrules pour quelques espèces.  
Source : figure générée depuis `notebooks/excel_eda.ipynb`.

Cette analyse joue un rôle important dans la conception de Polybase. Elle montre que l'application doit stocker les relevés dans un format plus structuré que les fichiers Excel. Elle montre aussi que l'import devra être validé avec l'équipe de l'Aquarium, car certaines décisions demandent une confirmation métier. Le notebook sert donc de pont entre les fichiers historiques et le futur import serveur.

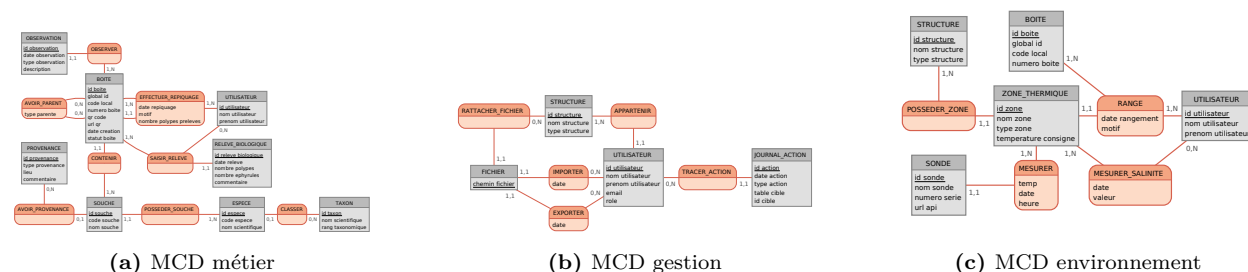
## 7. Conception de Polybase

### 7.1. Choix d'architecture

Le choix retenu est hybride. Django garde la base de données, les API, l'authentification, l'administration, les exports serveur et les pages liées à la confidentialité. React sert pour l'application principale, qui doit être plus fluide pour la tablette et plus agréable à utiliser sur ordinateur. Ce choix est cohérent avec le besoin observé dans le projet. Django est adapté aux modèles de données, aux permissions, aux formulaires serveur, à l'administration et aux exports. React est adapté aux écrans interactifs, à la recherche, aux suggestions, à la fiche boîte et aux adaptations entre tablette et ordinateur. Cette séparation évite de concentrer toute la logique dans une seule partie du projet. La configuration de base de données permet deux situations. En développement local, si aucune variable `POSTGRES_DB` n'est définie, Django utilise SQLite avec `backend/db.sqlite3`. Si les variables PostgreSQL sont présentes, Django utilise PostgreSQL via `psycopg`. Cette organisation permet de travailler simplement en local tout en gardant une configuration possible pour une base PostgreSQL.

### 7.2. Modèle de données

Le modèle de données est réparti dans plusieurs apps Django. Cette séparation rend le projet plus lisible et plus maintenable. Elle évite de concentrer tous les modèles dans une seule app générale. Chaque app correspond à un domaine fonctionnel. Le MCD présenté dans ce mémoire doit être lu comme une version de travail. Il a déjà été discuté et optimisé avec l'aide de Madame Sandra Bringay, afin de mieux clarifier les entités et les relations importantes. L'ancien MCD complet a été remplacé par trois volets : métier, gestion et environnement. Ces schémas doivent encore être repris proprement avant d'être considérés comme une version finale. Les figures ci-dessous conservent l'état disponible au moment de la rédaction, car elles permettent de comprendre la logique générale du modèle.



**Figure 8** – Version actuelle du modèle conceptuel de données.  
Sources : Réalisé avec mocodo.

Les organisations et partenaires sont gérés dans l'app **organizations**. Cette partie permet de représenter l'Aquarium de Paris et de futures structures partenaires. Les comptes, rôles et préférences sont gérés dans **accounts**. Les rôles présents sont administrateur, technicien laboratoire et lecteur. La taxonomie est séparée dans **taxonomy**, avec les espèces, les origines et les souches. Les boîtes, zones thermiques, mouvements, repiquages, tags et transferts sont regroupés dans **cultures**. Les relevés biologiques, observations, sondes, températures, salinités et anomalies sont regroupés dans **measurements**. Les imports, exports, alertes et journaux d'action sont prévus dans **exports** et **audit**. Ce découpage garde la boîte de culture comme point d'entrée principal. Une boîte est reliée à une organisation, une souche, une zone thermique, un historique de

relevés et des événements. Cette structure prépare l’usage en laboratoire et la consultation sur ordinateur. Le modèle prépare aussi l’usage entre structures. Le champ `global_code` identifie une boîte de façon stable, tandis que `local_code` permet de conserver une notation interne. Les organisations, accords de partage, transferts et tags d’identification donnent une base pour gérer les échanges futurs sans mélanger les données de chaque aquarium.

### 7.3. Interface et maquette

L’application doit répondre à deux situations très différentes. La première se passe en laboratoire, sur tablette. Dans ce contexte, l’interface doit aller droit au but. Elle doit permettre de scanner ou rechercher une boîte, de lire son dernier relevé et de saisir rapidement les valeurs du jour. L’écran doit rester léger, car l’utilisateur est en situation de manipulation et cherche une action rapide. La seconde situation se passe sur ordinateur. L’objectif est alors de consulter les données avec plus de recul, d’administrer les informations, de gérer les exports et de suivre les paramètres de l’application. La maquette papier, présentée en annexe, a servi à clarifier les usages sans imposer un rendu final. Elle a surtout permis d’identifier les parcours principaux : accéder rapidement à une boîte, saisir un relevé, revoir l’historique, consulter les zones thermiques et gérer le profil. La maquette Figma a été utilisée comme inspiration visuelle ponctuelle, puis le projet a évolué vers une interface différente, plus sobre et plus adaptée aux contraintes du laboratoire.

## 8. Réalisations techniques

### 8.1. Backend Django et API

Une première API propre a été mise en place avec Django REST Framework. Les anciens endpoints prototypes en français ont été retirés. Les endpoints actuels sont vérifiés par des tests. Le frontend doit utiliser les routes anglaises stables.

| Endpoint                           | Rôle                                                                                     |
|------------------------------------|------------------------------------------------------------------------------------------|
| GET /api/health/                   | Vérifie que le service répond. Cette route est publique.                                 |
| GET /api/dashboard/                | Renvoie les données de synthèse pour le premier écran React.                             |
| GET /api/boxes/                    | Renvoie la liste paginée des boîtes, avec filtres par recherche, statut et organisation. |
| GET /api/boxes/<id>/               | Renvoie le détail d'une boîte avec son historique de relevés.                            |
| GET /api/boxes/<id>/measurements/  | Renvoie les relevés biologiques d'une boîte.                                             |
| POST /api/boxes/<id>/measurements/ | Crée ou met à jour le relevé d'une boîte pour une date donnée.                           |
| GET /api/thermal-zones/            | Renvoie les zones thermiques, les sondes et les dernières mesures.                       |
| GET /api/profile/                  | Renvoie le profil utilisateur, ses organisations et sa langue.                           |
| PATCH /api/profile/                | Met à jour les préférences du compte, dont la langue d'interface.                        |

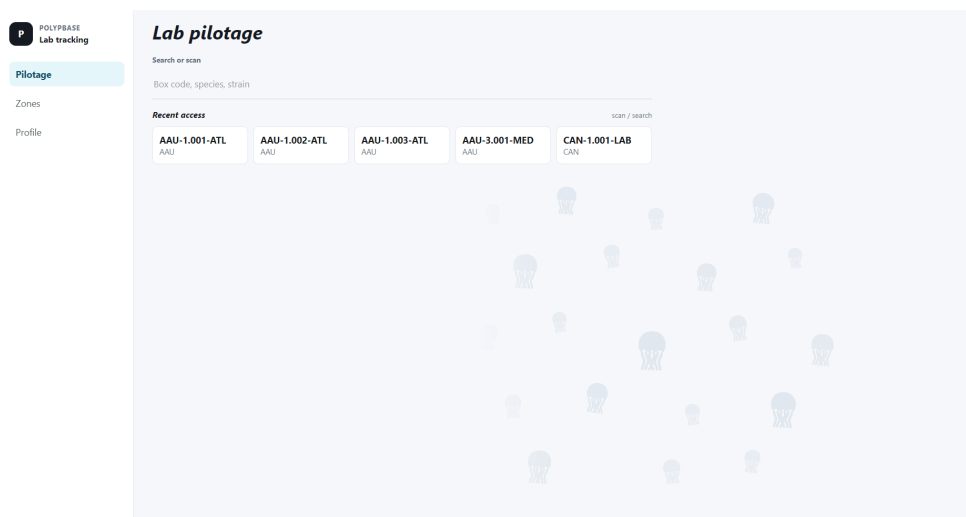
**Table 2** – Endpoints API présents dans la configuration Django.

Les permissions sont appliquées côté backend. La liste des boîtes utilise les organisations autorisées pour l'utilisateur. Le détail d'une boîte utilise le même filtrage. La création d'un relevé vérifie que l'utilisateur peut écrire des données de laboratoire. L'API refuse la création d'un relevé biologique pour un utilisateur en lecture seule. L'endpoint de création de relevé utilise `update_or_create`. Cela signifie qu'un relevé pour une même boîte et une même date est créé s'il n'existe pas, puis mis à jour s'il existe déjà. Une entrée de journal est ajoutée dans `AuditLog`. L'action est `ENTRY` lors d'une création et `UPDATE` lors d'une mise à jour.

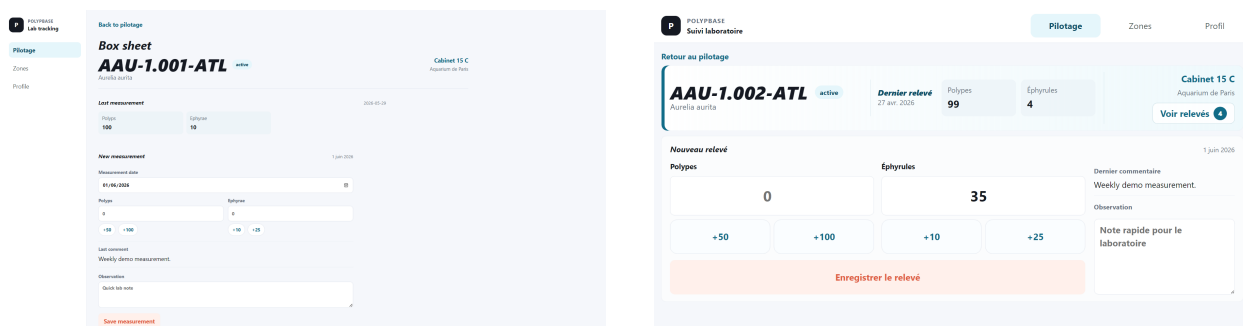
### 8.2. Interface React

L'interface React actuelle est une première version fonctionnelle. Elle sert à tester le lien entre les données du backend et les écrans principaux attendus. Le code se trouve principalement dans `frontend/src/App.tsx`, avec les types dans `frontend/src/types.ts`, le client API dans `frontend/src/api/client.ts` et le style dans `frontend/src/styles/app.css`. L'onglet pilotage sert de point d'entrée. Il propose une recherche par code, espèce ou souche. Il affiche aussi les derniers accès et des suggestions de boîtes. Lorsqu'une boîte est sélectionnée, l'application charge son détail via `/api/boxes/<id>/`. Cette logique permet d'aller rapidement vers la fiche d'une boîte sans passer par une liste longue. La fiche boîte affiche le code global, l'espèce, le statut, la zone thermique, l'organisation, le dernier relevé, le formulaire de nouveau relevé et l'historique. Le formulaire envoie les données au backend avec `apiPost`, puis recharge la fiche pour afficher les informations à jour. La version tablette fusionne l'identité de la boîte et le dernier relevé dans une bande haute plus compacte. Sur tablette, le bouton *Voir relevés* ouvre l'historique dans une fenêtre. L'onglet zones affiche les zones thermiques accessibles à l'utilisateur. Les données viennent de `/api/thermal-zones/`. Chaque zone contient son organisation, sa température cible, son statut, le nombre de boîtes, la dernière température, la dernière salinité et les sondes associées. L'onglet profil affiche l'utilisateur connecté, ses organisations et le

choix de langue.



**Figure 9** – Premier écran de pilotage dans l’interface React.  
Source : capture réalisée depuis l’interface React de Polybase.



**Figure 10** – Fiche boîte sur ordinateur et adaptation en paysage pour tablette.  
Source : captures réalisées depuis l’interface React de Polybase.

### 8.3. Traduction, données de test et outillage

Le système de traduction a été ajouté. Le français est la langue par défaut et l’anglais est prévu comme seconde langue. La préférence est modifiable dans le profil. Le projet contient les fichiers `.po` et `.mo`, ainsi qu’un script de compilation compatible avec l’environnement Windows du projet. Le projet contient aussi une commande `seed_demo_data` qui crée un jeu de données local de démonstration. Cette commande crée des utilisateurs, des organisations, des souches, des zones, des sondes, des boîtes, des relevés, des observations, des alertes, de la parenté, des transferts, des imports et des exports de démonstration. Les dépendances Python sont gérées avec `uv`. Ce choix facilite la reprise du projet, car les dépendances sont déclarées dans `pyproject.toml` et verrouillées dans `uv.lock`. Une personne qui récupère le projet peut recréer l’environnement avec `uv venv` et `uv sync`.

## 9. Confidentialité, durabilité et impact de l’outil

Polyibase a un impact organisationnel avant d’avoir un impact purement technique. L’application vise à réduire la dépendance à des fichiers Excel dispersés et compris par un nombre limité de personnes. Elle doit faciliter la transmission des connaissances, la traçabilité des actions et la reprise du suivi par de nouveaux membres du laboratoire. Cette dimension rejoint une logique de durabilité de l’information scientifique. La dimension DDRS apparaît aussi dans la conservation et la valorisation des données. Les cultures de polypes représentent un patrimoine biologique suivi dans le temps. Un outil structuré permet de mieux garder l’historique, de limiter les pertes d’information et de préparer des analyses futures. Le partage maîtrisé avec d’autres aquariums ou structures partenaires peut aussi favoriser la coopération scientifique, à condition de respecter des règles d’accès claires. La dimension RSE concerne la qualité de l’organisation du travail. Un outil plus clair peut réduire les erreurs de saisie, rendre les responsabilités plus explicites et donner aux utilisateurs une interface adaptée à leur situation réelle. Sur tablette, l’objectif est de réduire la charge cognitive pendant les manipulations. Sur ordinateur, l’objectif est de faciliter la consultation et l’administration. Le RGPD concerne surtout les comptes utilisateurs, les rôles, les préférences de langue et les journaux d’action. La CNIL rappelle les principes de finalité, de minimisation, de durée de conservation, de sécurité et de droits des personnes [8, 9]. Ces principes sont pris en compte dans le projet par une collecte limitée aux données nécessaires, une page de confidentialité, des rôles, des accès filtrés par organisation, une authentification Django et une traçabilité des actions. La page de confidentialité du projet indique que les données servent au suivi des cultures, à la gestion des comptes, à la sécurité, aux exports et à l’historique du laboratoire. Elle indique aussi que les utilisateurs peuvent demander l’accès, la rectification ou la suppression de leurs données de compte auprès de l’administrateur de leur organisation. En production, l’application devra être servie en HTTPS, avec des sauvegardes régulières et une base de données protégée. L’impact de Polyibase dépendra donc de sa qualité technique et de son adoption. Une application trop complexe risquerait de recréer les difficultés observées dans certains outils existants. Une application simple, documentée et centrée sur les usages du laboratoire peut au contraire devenir un support durable pour le suivi des cultures.

## **10. Bilan critique et perspectives**

### **10.1. Limites actuelles**

Le premier recul concerne le modèle de données. Le MCD a déjà été retravaillé avec l'aide de Madame Sandra Bringay, et il doit encore être refait proprement. Cette étape est importante, car une application de suivi biologique dépend fortement de la qualité de son modèle. Une mauvaise séparation entre boîte, souche, espèce, observation, mesure et événement rendrait la maintenance plus difficile. Le deuxième recul concerne l'import Excel. Le notebook montre qu'une transformation en format long est possible. Le script d'import réel reste à écrire. Cette étape demande une validation métier, notamment pour les cellules vides, les changements de température, les repiquages et les feuilles complémentaires. Le risque principal serait de transformer trop vite les fichiers sans comprendre ce que certaines absences de valeur signifient.

### **10.2. Recul sur l'interface et les capteurs**

Le troisième recul concerne l'interface. La version React actuelle permet de tester les premiers usages. Elle devra encore être testée sur le format matériel visé, en particulier sur tablette Samsung Galaxy Tab Active 5 en orientation paysage. Il faudra vérifier la lisibilité, la taille des boutons, la saisie tactile, la navigation et le comportement réel en laboratoire. Le quatrième recul concerne les sondes. Les modèles sont prêts et la note d'intégration iMinilide-A donne une direction. La logique d'interrogation reste à écrire. Le format exact du XML devra être obtenu en inspectant la réponse réelle de la sonde physique. La finalisation du parser dépend de ces informations.

### **10.3. Perspectives**

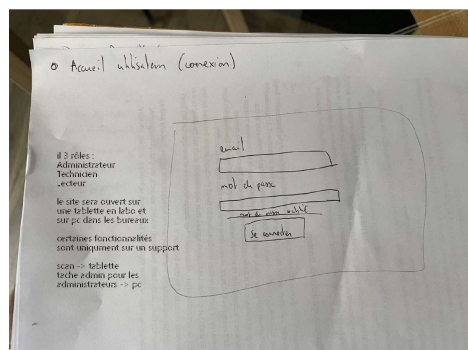
Les perspectives principales sont donc l'import Excel réel, la génération des QR codes, les exports filtrés, l'intégration des sondes, l'administration métier, les tests tablette et la consolidation du MCD. Ces étapes devront garder une même exigence : éviter les déductions fragiles, valider les règles métier avec l'Aquarium et conserver un code suffisamment clair pour être repris après le stage.

## 11. Conclusion

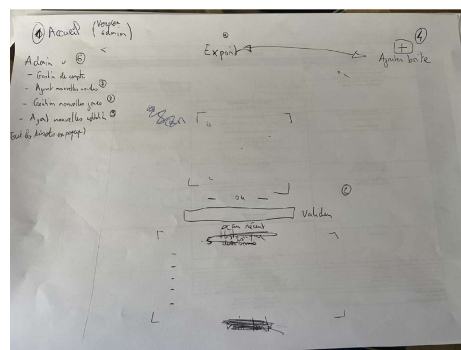
Polyibase est passé du cadrage initial à une base technique exploitable. Le projet contient une architecture hybride claire, un backend Django structuré, une API propre, des modèles métier cohérents, une première interface React, une gestion des langues, une page de confidentialité, des données de démonstration et un notebook d'analyse exploratoire des fichiers Excel. Le travail réalisé pose les fondations d'une application encore en construction. Les choix faits vont dans le sens d'un outil maintenable : les domaines métier sont séparés en apps Django, l'API utilise Django REST Framework, React est réservé à l'interface principale, les dépendances Python sont gérées avec `uv` et les données réelles restent hors du versionnement. Le projet a aussi permis de mieux comprendre la relation entre besoin métier, données historiques et conception logicielle. Dans ce stage, la difficulté principale vient de la précision du contexte. L'interface compte beaucoup, car elle doit être utilisée vite et souvent. Le modèle de données compte tout autant, car il doit garder la trace des cultures vivantes, des événements, des organisations et des échanges futurs entre aquariums. La suite du stage devra consolider cette base pour rendre Polyibase réellement utilisable au laboratoire et partageable avec d'autres structures.

## A Maquette papier

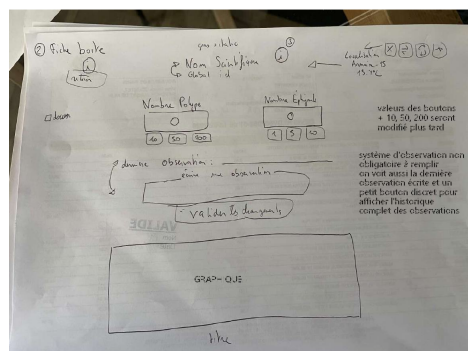
La maquette papier a servi à clarifier les parcours avant de stabiliser l'interface React. Elle présente les premières idées d'écrans autour du pilotage, de la fiche boîte, de la saisie d'un relevé, de l'historique et des zones thermiques. Elle n'est pas utilisée comme rendu final. Elle reste utile pour comprendre le cheminement entre le besoin exprimé au laboratoire et les premiers choix d'interface.



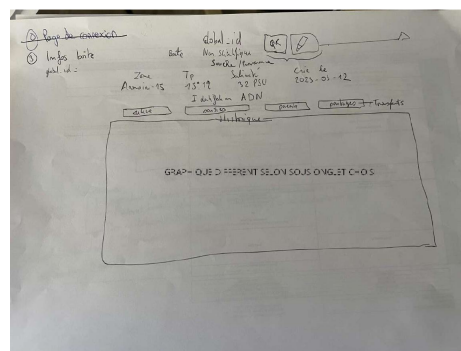
(a) Page 1



(b) Page 2

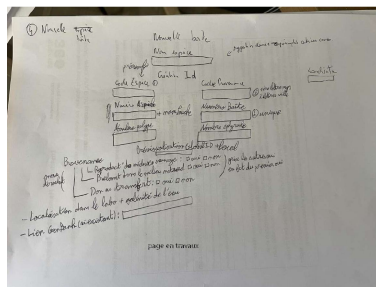


(c) Page 3

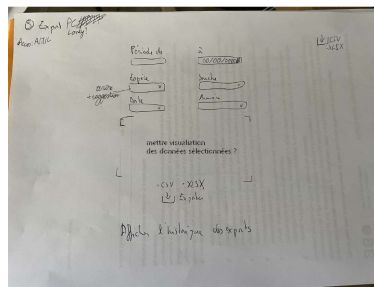


(d) Page 4

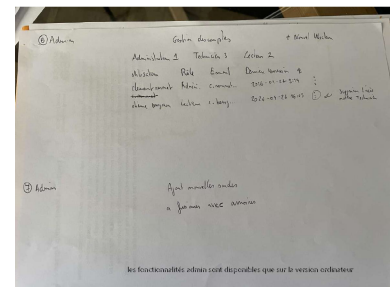
**Figure 11** – Premières pages de la maquette papier.  
Source : docs/maquette\_papier/maquette.pdf.



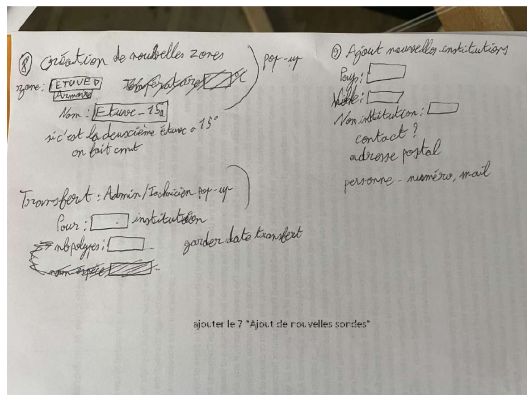
(a) Page 5



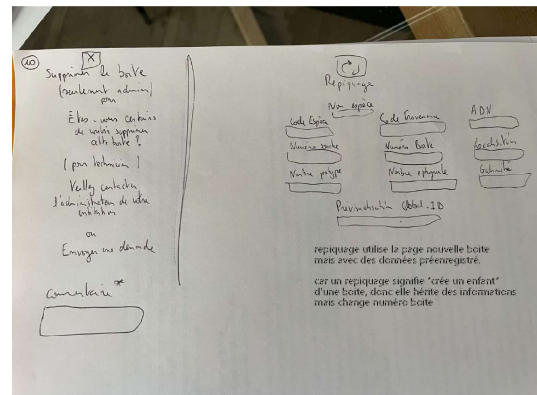
(b) Page 6



(c) Page 7



(d) Page 8



(e) Page 9

**Figure 12** – Pages complémentaires de la maquette papier.  
Source : docs/maquette\_papier/maquette.pdf.

## Références

- [1] Aquarium de Paris. *Médusarium*. <https://aquariumdeparis.com/tours/medusarium/>.
- [2] Jellyfish and Polyps Academy. *Laboratoire*. <https://jellyfishandpolypsacademy.com/laboratoire/>.
- [3] Luxaqua. *LXA Fauniabase*. <https://www.luxaqua.tech/fr/fauniabase/>.
- [4] World Register of Marine Species. *About WoRMS*. <https://www.marinespecies.org/about.php>.
- [5] Helm, R. R. *Evolution and development of scyphozoan jellyfish*. Biological Reviews, 2018. <https://pubmed.ncbi.nlm.nih.gov/29446223/>.
- [6] Fuchs, B. et al. *Regulation of polyp-to-jellyfish transition in Aurelia aurita*. Current Biology, 2014. <https://doi.org/10.1016/j.cub.2013.12.003>.
- [7] Treible, L. M. et Condon, R. H. *Temperature-driven asexual reproduction and strobilation in three scyphozoan jellyfish polyps*. Journal of Experimental Marine Biology and Ecology, 2019. <https://doi.org/10.1016/j.jembe.2019.151204>.
- [8] CNIL. *Les six grands principes du RGPD*. <https://www.cnil.fr/fr/comprendre-le-rgpd/les-six-grands-principes-du-rgpd>.
- [9] CNIL. *Les durées de conservation des données*. <https://www.cnil.fr/fr/passer-laction/les-durees-de-conservation-des-donnees>.